COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	07/24/26	**DATE SUBMITTED:**	07/24/26
NAME:	Del Mundo, Kenneth James C.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. _3__ [ACTIVITY NAME] Linked list

1. PROCEDURE OUTPUT

What procedures did you perform?

```cpp
#include <iostream>
#include "doubly_ll.h"

using namespace std;

int main()
{
    // Create the nodes: C P E
    DoubleList<char>* head = createNode('C');
    DoubleList<char>* second = createNode('P');
    DoubleList<char>* third = createNode('E');

    // Connect the nodes

    // Connect the head node to the next node
    head->next = second;
    second->prev = head;

    second->next = third;
    third->prev = second;

    // Traversal
    cout << "Testing dllTraverse" << endl;
    dllTraverse(head);

    // Insert at Head
    cout << "\nTesting dllInsertHead" << endl;
    dllInsertHead('G', &head);
    dllTraverse(head);

    // Insert at End
    cout << "\nTesting dllInsertEnd" << endl;
    dllInsertEnd('0', head);
    dllInsertEnd('1', head);
    dllInsertEnd('0', head);
    dllTraverse(head);

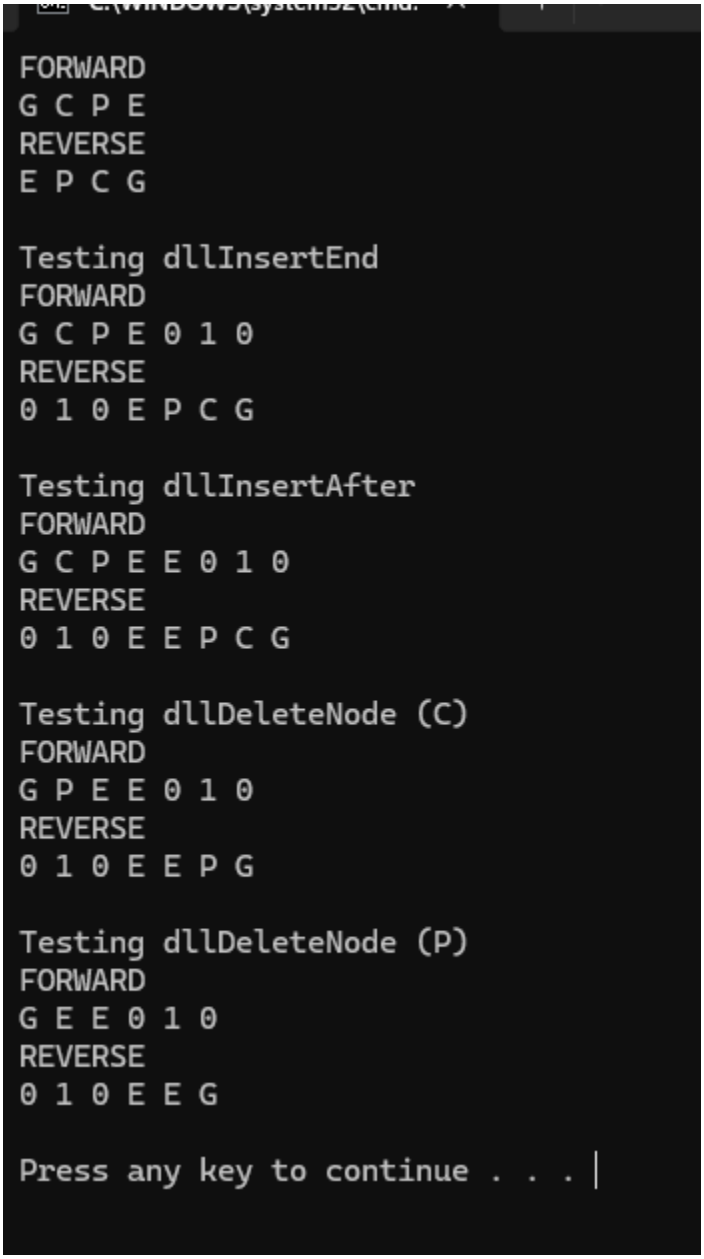
    // Insert after P
    cout << "\nTesting dllInsertAfter" << endl;

    DoubleList<char>* current = head;
    while (current != nullptr && current->data != 'P')
        current = current->next;

    if (current != nullptr)
        dllInsertAfter(current, 'E');
```

```
48
49      dllTraverse(head);
50
51      // Delete C
52      cout << "\nTesting dllDeleteNode (C)" << endl;
53      dllDeleteNode(&head, 'C');
54      dllTraverse(head);
55
56      // Delete P
57      cout << "\nTesting dllDeleteNode (P)" << endl;
58      dllDeleteNode(&head, 'P');
59      dllTraverse(head);
60
61      return 0;
62  }
63
```

OUTPUT:


FORWARD
G C P E
REVERSE
E P C G
Testing dllInsertEnd
FORWARD
G C P E 0 1 0
REVERSE
0 1 0 E P C G
Testing dllInsertAfter
FORWARD
G C P E E 0 1 0
REVERSE
0 1 0 E E P C G
Testing dllDeleteNode (C)
FORWARD
G P E E 0 1 0
REVERSE
0 1 0 E E P G
Testing dllDeleteNode (P)
FORWARD
G E E 0 1 0
REVERSE
0 1 0 E E G
Press any key to continue . . .

doubly_ll.h > createNode<T>(T)

```cpp
#ifndef DOUBLY_LL_H
#define DOUBLY_LL_H

#include <iostream>
using namespace std;

template <typename T>
class DoubleList {
public:
    T data;
    DoubleList<T>* next;
    DoubleList<T>* prev;
};

// Create a node function
template <typename T>
DoubleList<T>* createNode(T newData) {
    //dynamic memory allocation for a new code
    DoubleList<T>* newNode = new DoubleList<T>;

    newNode->data = newData;
    newNode->next = nullptr;
    newNode->prev = nullptr;

    return newNode;
}

// Traverse
template <typename T>
void dllTraverse(DoubleList<T>* currentNode) {

    if (currentNode == nullptr) {
        cout << "The list is empty." << endl;
        return;
    }

    DoubleList<T>* tail = nullptr;

    cout << "FORWARD" << endl;

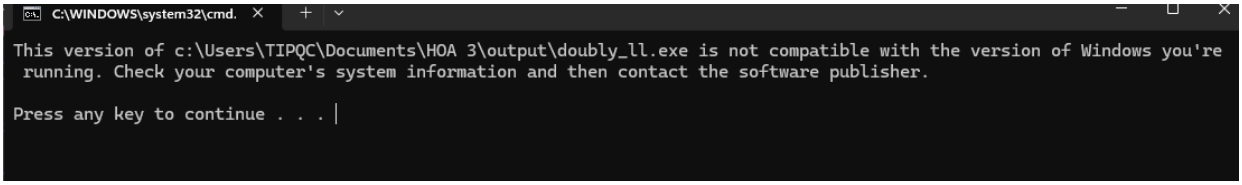
    while (currentNode != nullptr) {
        cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }

    cout << endl;
```

```
48
49      cout << "REVERSE" << endl;
50
51      while (tail != nullptr) {
52          cout << tail->data << " ";
53          tail = tail->prev;
54      }
55
56      cout << endl;
57  }
58
59  // Insert at Head
60  template <typename T>
61  void dllInsertHead(T newData, DoubleList<T>** currentNode) {
62
63      DoubleList<T>* newNode = createNode(newData);
64
65      newNode->next = *currentNode;
66
67      if (*currentNode != nullptr)
68          (*currentNode)->prev = newNode;
69
70      *currentNode = newNode;
71  }
72
73  // Insert at End
74  template <typename T>
75  void dllInsertEnd(T newData, DoubleList<T>* currentHead) {
76
77      DoubleList<T>* newNode = createNode(newData);
78
79      while (currentHead->next != nullptr)
80          currentHead = currentHead->next;
81
82      currentHead->next = newNode;
83      newNode->prev = currentHead;
84  }
85
86  // Insert After
87  template <typename T>
88  void dllInsertAfter(DoubleList<T>* previous, T newData) {
```

```
89
90      if (previous == nullptr)
91          return;
92
93      DoubleList<T>* newNode = createNode(newData);
94
95      newNode->next = previous->next;
96      newNode->prev = previous;
97
98      if (previous->next != nullptr)
99          previous->next->prev = newNode;
100
101     previous->next = newNode;
102 }
103
104 // Delete Node
105 template <typename T>
106 void dllDeleteNode(DoubleList<T>** head, T key) {
107
108     DoubleList<T>* temp = *head;
109
110     while (temp != nullptr && temp->data != key)
111         temp = temp->next;
112
113     if (temp == nullptr)
114         return;
115
116     if (temp->prev != nullptr)
117         temp->prev->next = temp->next;
118     else
119         *head = temp->next;
120
121     if (temp->next != nullptr)
122         temp->next->prev = temp->prev;
123
124     delete temp;
125 }
126
127 #endif
```

OUTPUT:


C:\WINDOWS\system32\cmd.
This version of c:\Users\TIPQC\Documents\HOA 3\output\doubly_ll.exe is not compatible with the version of Windows you're running. Check your computer's system information and then contact the software publisher.
Press any key to continue . . .

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.